

RF-One - Server Performance & Customer Intelligence

Concept for Review | Shelbi, Anthony & Tatiana

First: What is RF-One?

RF-One is an artificial intelligence system I am developing, using Rome's Flavours as the first real operating environment.

The basic idea is simple: a restaurant already generates a large amount of information - Clover, sales, shifts, payroll, purchasing, tips, employees, customers - but today those data usually remain separated and are used mainly to produce reports.

RF-One is intended to understand what is actually happening in the restaurant, compare it with what we want to achieve, and use that knowledge to help people make better decisions and continuously improve.

It is therefore not simply reporting software, and it is not intended to replace managers. The goal is to create a permanent operational intelligence for the restaurant: something that observes data and results, learns over time, and helps management and staff in everyday work.

We are already developing several separate but connected modules, including Sales, Purchasing, Payroll, Tips, Organization, and others.

This document focuses specifically on the work of servers. I do not want to build another system that merely says, 'This server sold more than that server.'

The goal is to understand how each individual server works, where that person is strong, where opportunities are being lost, how performance changes under pressure, and above all how RF-One can help that person become better and earn more money.

From this discussion, a second concept also emerged: Customer Intelligence, designed to understand the consumption behavior of each table and, when possible, each identifiable customer over time.

The two work together: Customer Intelligence tries to understand the table and the opportunities available. Server Performance tries to understand how well the server uses those opportunities and how to help the server improve.

What follows is not a final specification. It is a concept I want you to challenge from practical experience on the floor.

1. Server Performance: Not a Ranking, but a Growth System

The objective is not simply to determine who sells the most. Every server is different in experience, capability, character, motivation, and personal circumstances. RF-One should not try to guess why someone performed better or worse on a particular day.

Instead, it should learn over time:

- where that person is strong;
- where opportunities are being lost;
- how that person reacts under pressure;

- how performance evolves;
- which suggestions or interventions actually help that person improve.

Each server should be evaluated against two references at the same time:

Brand Expectation

What the restaurant wants the service to achieve.

Personal Baseline

What that specific person normally achieves.

This allows RF-One to distinguish, for example, between someone who is still below the Brand standard but improving rapidly and someone who looks strong compared with the team but is actually declining relative to his or her own baseline.

2. What We Really Want to Monitor

Performance is not simply Sales.

Productivity

How much economic value the server produces relative to time worked and opportunities available. A first example is Sales per Hour Worked. But that number alone is not enough.

Quality of Sale

What matters is not only how much is sold, but what is sold. A server can handle many tables and generate good revenue simply by taking orders quickly, while still leaving value on the table.

RF-One should therefore observe, for example:

- appetizers;
- wine and drinks;
- desserts;
- premium items;
- add-ons and modifiers;
- products the Brand wants to promote;
- the overall composition of the sale.

A server can therefore have strong productivity and weak Quality of Sale.

Opportunity Capture

The question changes from 'Did you sell wine?' to:

Given the characteristics of this particular table, what opportunities were realistically available, and how many of them were captured?

It would make no sense to penalize a server for not selling wine to a table that clearly does not want it. RF-One must progressively learn the realistic potential of different types of tables.

The useful comparison becomes Available Opportunity vs. Captured Opportunity and, where possible, Potential Value vs. Value Actually Captured.

Operational Quality

High sales do not automatically mean high performance. RF-One should also observe evidence such as discounts, voids, refunds, errors, unusual adjustments, payment problems, and other operational anomalies. These are not automatically the server's fault, but they are evidence that must be interpreted with the rest of the picture.

Perceived Service Quality

There is also something Clover cannot tell us well enough: how the guest actually perceived the service. A possible future solution is a very short survey accessed through a QR code at the table. Table, time, and Clover data could link the response automatically to the actual service received, so the guest does not have to re-enter information the system already knows. The questions should be very few and very targeted.

3. Performance Under Pressure

A particularly important variable is the number of guests being served concurrently, not the total number served across the entire shift.

What happens to this server's performance as simultaneous service load increases?

We want to understand at least three characteristics:

Capacity

How much concurrent service demand the server can manage effectively.

Acceleration

How effectively the server can increase pace as load grows.

Resilience

How well the server maintains Quality of Sale, service quality, and operational discipline while pressure increases.

A server might be excellent with 10 simultaneous guests but deteriorate sharply at 20. Another might be less impressive at low load but maintain quality almost unchanged when the restaurant is full.

Low load is informative too. With fewer simultaneous tables, the server has more time to care for the guest, explain products, make targeted recommendations, and develop the sale more fully.

4. From Control to Coaching

RF-One should not stop at telling a server, 'You are underperforming here.' It should try to help that person improve.

Every intervention should use two complementary levers:

Pride / Recognition

Show what the person does well, improvements, strengths, achieved results, and completed challenges.

Personal Economic Benefit

Whenever reasonably possible, translate the improvement into potential additional tip dollars for the server.

If you moved your dessert attach rate from 10% to 16%, given your typical volume, that could mean approximately \$X in additional sales and about \$Y more in tips per week.

These are estimates, not promises. The message should be simple: improving the quality of the restaurant's sales can also put more money directly into the server's pocket.

5. Personalized Coaching

RF-One should learn not only what a person needs to improve, but how to help that specific person in the way that works best.

Coaching should combine three sources:

Brand Playbook

How we want service to be delivered and how products and opportunities should be presented.

Best Performers

Behaviors that, in the restaurant's real data, appear to produce particularly strong outcomes.

Individual Learning

How that particular person responds to different interventions. One server may respond best to numerical goals, another to recognition, challenges, competition, or very concrete behavioral suggestions.

RF-One should progressively learn which coaching style actually produces results for each person.

6. Service Copilot

Server Performance feeds a second capability: the Service Copilot. The idea is for RF-One to support the server during the actual service, not only before or after it.

Before Service

A short personal briefing - not a page of numbers. One priority, one strength to leverage, one objective, one particular opportunity, and when possible the potential personal financial upside.

During Service

Very short, table-specific suggestions, for example:

- Table 12 - wine opportunity
- Table 8 - dessert moment
- Table 4 - returning guest / known preference
- High load - use short recommendation

RF-One must learn two different things:

Next Best Action

What is the best thing to do now?

Next Best Moment

Is this the right moment to suggest it? A correct recommendation delivered at the wrong time can be useless or even harm the service. Sometimes the best decision should simply be: do not interrupt the server now.

After Service

Very short feedback: what went well, what improved, opportunities missed, progress versus Personal Baseline, distance from Brand Expectation, possible effect on tips, and one concrete focus for the next shift.

7. How Intrusive Should RF-One Be?

Management should decide this. The restaurant must be able to control how much RF-One intervenes.

Observe

RF-One analyzes only.

Recommend to Manager

RF-One proposes an intervention to management.

Coach with Approval

RF-One prepares the suggestion, but a manager approves it.

Autonomous Coaching

RF-One communicates directly with the server within predefined limits.

The principle is simple: RF-One may become very present, but management controls how present it is allowed to be.

8. Company Smartwatch

During service, we see a company smartwatch as a potentially ideal Service Copilot interface. It should be discreet, extremely fast, hands-free, and minimally intrusive.

The watch should not show dashboards. It should show only information that can be acted on immediately.

Table 12 - Wine opportunity

Table 8 - Dessert moment

The watch can also become something more interesting: a human sensor.

9. The Server Adds What Systems Cannot Know

With a single tap, the server could add context such as:

- first time;
- regular;
- celebration;
- in a hurry;
- problem / waiting;
- not interested;
- special request;
- very satisfied.

These signals can immediately change how RF-One interprets the table. But the rule must be strict: micro-input, never data entry. If the server has to start typing during service, we have designed the wrong system.

10. Customer Intelligence - A Separate Module

During this discussion, a second module emerged that is broader than Server Performance. For now, we are calling it Customer Intelligence.

Its job is to progressively understand: How is this table consuming? And, when the customer can be identified: How does this customer tend to consume over time?

This module should remain separate because the intelligence it creates can support many other areas of RF-One.

11. Dining Session Profile

For each visit, RF-One can progressively build a Dining Session Profile using observations such as:

- number of guests;
- daypart;
- day of week;
- service duration;
- food families;
- drink families;
- carb/protein balance;
- appetizers;
- wine/drinks;
- premium vs. basic items;
- modifiers/add-ons;
- food-drink combinations;
- order sequence;
- spending range;
- check progression.

The important point is that this profile is not static. At seating we know very little. After the first drink we know something. After appetizers and main courses we know much more. RF-One can therefore continuously update its interpretation of the table and the remaining opportunities.

12. Food and Drink Correlations

Items sold can reveal useful patterns, for example:

- which drinks are ordered with particular food families;
- which appetizers increase the probability of certain main courses;
- which combinations are more likely to lead to dessert;
- which protein-heavy consumption patterns correlate with certain beverages;
- which add-ons are purchased together with particular dishes.

When this type of consumption pattern appears, which complementary products have the highest probability of being accepted?

That knowledge can directly feed the opportunities suggested by the Service Copilot.

13. Customer Consumption Profile

When a customer can be identified through reservation systems, CRM, or other sources, RF-One can build a longitudinal profile over time.

It could progressively learn food preferences, drink preferences, spending range, preferred combinations, types of visits, recommendations normally accepted, products normally rejected, and recurring patterns.

But RF-One must always keep a fundamental distinction:

Observed

What we actually know. Example: the guest ordered Chianti on six previous visits.

Derived

What can be calculated directly from observed facts.

Inferred

What RF-One considers probable. Example: the guest appears to prefer structured red wines.

An inference must never silently become a fact.

14. Where Customer Information Can Come From

Customer Intelligence should not depend on a single provider. In the future, it may receive evidence from OpenTable, Resy, other reservation systems, CRM platforms, Clover, guest surveys, server inputs, and visit history.

The objective is to build an RF-One representation of the customer and the Dining Session that remains independent of any one provider.

15. How Customer Intelligence Supports Other Modules**Server Performance**

Helps determine which opportunities were realistically available to the server.

Service Copilot

Helps suggest the best proposal at the best moment.

Training

Allows targeted training. Not simply 'sell more wine,' but 'with this type of table and this type of food, these approaches work better.'

Menu

Helps understand which products are purchased together, which categories complement each other, which sequences work, and which products generate other sales.

Marketing

Enables segmentation based on actual consumption behavior rather than only demographics or campaigns.

Sales Analytics

Helps understand not only what we sell, but how, when, and together with what it is purchased.

Purchasing / Inventory

Can progressively improve understanding of demand patterns, consumption combinations, and correlated sales behavior.

Brand

Can help determine whether actual customer and server behavior is consistent with the experience and positioning the Brand wants to create.

16. Training Changes Its Nature

With Service Copilot and Customer Intelligence, Training also changes. We no longer necessarily need to teach everything first and hope the server remembers it during service.

We can progressively teach the right thing at the moment it is needed.

A new server may receive substantial support. An experienced server may receive only high-value suggestions or help in unusual situations. As RF-One sees that a behavior has been internalized, assistance can be reduced.

This can reduce training time, accelerate development of new servers, make service more consistent, and reduce dependence on the quality of any single trainer or manager.

17. Brand Consistency Across Locations

This is one of the most important aspects. Winter Park and Mount Dora may have different people, different experience levels, different managers, and different moments. But if RF-One supports service according to the same Brand principles, Rome's Flavours can deliver a more recognizable experience regardless of location.

If the Brand grows, this becomes even more important. Service Copilot can help transform the Brand from something we explain to people into something that actually lives during service.

18. What RF-One Must Not Do

We do not want to start with: 'Clover gives us these data. Which KPIs can we build?'

The process must be the opposite:

First decide what we want to understand about people, service, and customers. Then identify what evidence allows us to observe it.

Clover will be an extremely important evidence source, but Clover must not define what it means to be a great server.

19. What Deliberately Stays Outside This Module

This module does not manage table assignment or floor distribution. Concurrent server load is important for understanding performance, but Server Performance does not decide which server receives the next table. That remains a separate process.

20. A Future Development: Judgment / Customer Reading

Later, we may introduce a dimension called Judgment / Customer Reading: how well a server understands when to push, when to change the proposal, when to stop, and when an RF-One suggestion is not appropriate.

The server could respond quickly with signals such as Not appropriate, Customer not interested, or Already declined. This should not automatically count as a missed opportunity.

RF-One should instead learn from the server's judgment as well.

RF-One teaches the server, but the server also teaches RF-One something about the reality of the customer.

This is a future phase and is not required for the first version.

Feedback Requested

I am especially interested in your practical opinion. I do not want you simply to tell me whether you 'like it.' I would like to know:

- What seems genuinely useful?

- What do you think would not work on the floor?
- What would be too intrusive?
- Which behaviors distinguish an excellent server from a mediocre one but are not measured today?
- Which commercial opportunities are normally lost?
- Which suggestions during service would actually help?
- Which suggestions would be irritating or useless?
- What table information would realistically be provided with a single tap?
- What do you think about the smartwatch concept?
- What is completely missing from this vision?

This is not a final specification.

This is exactly the moment when I want your real-world experience to modify it, correct it, and improve it.

RF-One | Concept for internal review